

React moderno na prática

Módulo: 09

Aula: 08 - Navegação e consumo de API com `fetch` e `axios`

Instrutor: Lucca Dumas

Duração: 2 h

CONTEÚDO

1) Abertura da aula

Esta aula integra **navegação entre telas** e **requisições HTTP** ao que já vimos no Módulo 09 (estado, efeitos e Context). O fluxo típico de uma tela com dados remotos é:

1. navegar entre telas;
2. buscar dados de API;
3. tratar carregamento e erro;
4. renderizar resultado na interface.

Os exemplos reforçam navegação com React Router e consumo de API com `fetch` e `axios`.

2) O que veremos hoje?

1. Navegação com `react-router-dom` (rotas e links)
2. Consumo de API com `fetch`
3. Consumo de API com `axios`
4. Estado de carregamento e erro
5. Hands-on com duas telas e uma lista simples

3) Navegação com React Router

Estrutura sugerida

```
src/  
  App.tsx  
  main.tsx  
  pages/  
    Home.tsx  
    UsuariosFetch.tsx  
    UsuariosAxios.tsx
```

`main.tsx` COM `BrowserRouter`

```
import { StrictMode } from "react";  
import { createRoot } from "react-dom/client";  
import { BrowserRouter } from "react-router-dom";  
import App from "./App";  
  
createRoot(document.getElementById("root")!).render(  
  <StrictMode>  
    <BrowserRouter>  
      <App />  
    </BrowserRouter>  
  </StrictMode>,  
);
```

`App.tsx` com rotas

```
import { Link, Route, Routes } from "react-router-dom";
import { Home } from "../pages/Home";
import { UsuariosFetch } from "../pages/UsuariosFetch";
import { UsuariosAxios } from "../pages/UsuariosAxios";

export default function App() {
  return (
    <main>
      <nav>
        <Link to="/">Home</Link> |{" "}
        <Link to="/usuarios-fetch">Usuarios (fetch)</Link> |{" "}
        <Link to="/usuarios-axios">Usuarios (axios)</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/usuarios-fetch" element={<UsuariosFetch />} />
        <Route path="/usuarios-axios" element={<UsuariosAxios />} />
      </Routes>
    </main>
  );
}
```

Home.tsx

```
export function Home() {
  return <h2>Home</h2>;
}
```

4) Consumo de API com `fetch`

UsuariosFetch.tsx

```

import { useEffect, useState } from "react";

type Usuario = {
  id: number;
  name: string;
  email: string;
};

export function UsuariosFetch() {
  const [usuarios, setUsuarios] = useState<Usuario[]>([]);
  const [carregando, setCarregando] = useState(false);
  const [erro, setErro] = useState("");

  useEffect(() => {
    async function carregar() {
      try {
        setCarregando(true);
        setErro("");

        const resposta = await fetch("https://jsonplaceholder.typicode.com/users");
        if (!resposta.ok) {
          throw new Error("Falha ao buscar usuarios");
        }

        const dados = (await resposta.json()) as Usuario[];
        setUsuarios(dados);
      } catch (e) {
        setErro("Nao foi possivel carregar usuarios");
      } finally {
        setCarregando(false);
      }
    }

    carregar();
  }, []);

  if (carregando) return <p>Carregando...</p>;
  if (erro) return <p>{erro}</p>;

  return (
    <section>
      <h2>Usuarios com fetch</h2>
      <ul>
        {usuarios.map((u) => (
          <li key={u.id}>
            {u.name} - {u.email}
          </li>
        ))}
      </ul>
    </section>
  );
}

```

5) Consumo de API com `axios`

`UsuariosAxios.tsx`

```

import { useEffect, useState } from "react";
import axios from "axios";

type Usuario = {
  id: number;
  name: string;
  email: string;
};

export function UsuariosAxios() {
  const [usuarios, setUsuarios] = useState<Usuario[]>([]);
  const [carregando, setCarregando] = useState(false);
  const [erro, setErro] = useState("");

  useEffect(() => {
    async function carregar() {
      try {
        setCarregando(true);
        setErro("");

        const resposta = await axios.get<Usuario[]>("https://jsonplaceholder.typicode.com/users",
        );
        setUsuarios(resposta.data);
      } catch (e) {
        setErro("Nao foi possivel carregar usuarios");
      } finally {
        setCarregando(false);
      }
    }

    carregar();
  }, []);

  if (carregando) return <p>Carregando...</p>;
  if (erro) return <p>{erro}</p>;

  return (
    <section>
      <h2>Usuarios com axios</h2>
      <ul>
        {usuarios.map((u) => (
          <li key={u.id}>
            {u.name} - {u.email}
          </li>
        ))}
      </ul>
    </section>
  );
}

```

6) fetch X axios (visão prática)

Ponto	fetch	axios
Origem	API nativa do navegador	Biblioteca externa
Parse de JSON	precisa chamar <code>res.json()</code>	já retorna em <code>response.data</code>
Status de erro HTTP	precisa validar <code>response.ok</code>	erros HTTP já caem no <code>catch</code>

Usar no projeto	útil quando não se quer dependência	padrão comum em camadas de
Fetch	fetch	axios

7) Boas práticas desta etapa

- sempre tratar **carregando**, **erro** e **sucesso**;
- tipar a resposta (`type Usuario` , etc.);
- evitar lógica de API diretamente espalhada em muitos componentes;
- usar `key` estável ao renderizar listas;
- manter URL de API centralizada quando o projeto crescer.

8) Hands-on da aula

Objetivo do hands-on

1. Montar navegação entre 3 páginas;
2. Implementar uma página com `fetch`;
3. Implementar outra com `axios`;
4. Exibir lista de usuários com carregamento e erro.

Passos

1. Configurar `BrowserRouter` em `main.tsx`.
2. Criar rotas no `App.tsx`.
3. Criar `Home.tsx`.
4. Criar `UsuariosFetch.tsx`.
5. Criar `UsuariosAxios.tsx`.
6. Rodar:

```
yarn dev
yarn lint
```

9) Erros comuns desta aula

1. **Esquecer `BrowserRouter` no `main.tsx`**
Resultado: erro de roteamento.
2. **Não validar `response.ok` no `fetch`**
Resultado: falhas HTTP podem passar sem tratamento correto.
3. **Não tratar estado de carregamento**
Resultado: tela parece travada durante a requisição.
4. **Não tratar `catch`**
Resultado: usuário não recebe feedback em caso de erro.
5. **Usar índice do array como `key` sem necessidade**
Resultado: comportamento inconsistente em listas.

10) Exercícios de fixação

1. Qual componente habilita navegação por rota no React Router?
 - ☐ `RouteSwitch`
 - ☒ `BrowserRouter`
 - ☐ `RouterProvider` obrigatório em todo cenário
 - ☐ `NavigateOnly`
2. No `fetch` , para checar erro HTTP antes de usar JSON, normalmente verificamos:
 - ☐ `response.data`
 - ☒ `response.ok`
 - ☐ `response.headers.error`
 - ☐ `response.route`
3. No `axios` , os dados retornados ficam em:
 - ☐ `response.json()`

- ☒ `response.data`
 - ☐ `response.payload`
 - ☐ `response.content`
4. Qual estado melhora a experiência durante chamada de API?
- ☐ `tema`
 - ☒ `carregando`
 - ☐ `contador`
 - ☐ `ativo`
5. Qual combinação representa o fluxo básico de uma tela com API?
- ☐ Apenas sucesso
 - ☒ carregando + erro + sucesso
 - ☐ apenas erro
 - ☐ erro + tema
-

11) Desafio para casa

1. Criar uma página `PostsFetch.tsx` buscando posts em:
 - `https://jsonplaceholder.typicode.com/posts`
2. Mostrar apenas:
 - `id`
 - `title`
3. Repetir a página com `axios` (`PostsAxios.tsx`).
4. Adicionar as duas rotas no menu.
5. Rodar:

```
yarn lint
```

Regras

- manter tipagem dos dados;
 - tratar carregamento e erro nas duas páginas;
 - manter componentes curtos e com responsabilidade clara.
-

12) Resumo final

Ao final desta aula, o aluno deve conseguir:

- montar navegação básica entre páginas com `react-router-dom` ;
- consumir API com `fetch` e com `axios` ;
- tratar carregamento e erro de forma explícita;
- renderizar listas com dados remotos;
- comparar as duas abordagens de requisição HTTP com critérios práticos.